



---

**POLYTECHNIQUE MONTRÉAL**

Département de génie électrique

*Hiver 2026*

---

# Rapport de projet final

ELE3000 – Projet personnel en génie électrique

*Simulation en temps réel de convertisseurs DC-DC sur FPGA*

**Rapport présenté à :**

M. Tarek Ould-Bachir, ing.

**Travail remis par :**

Ben Salem, Khalil

Matricule : 2231998

Dernière mise à jour : 24 avril 2026

## Table des matières

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Identification du problème</b>                                | <b>3</b>  |
| 1.1      | Définition du problème . . . . .                                 | 3         |
| 1.2      | Revue de la littérature et de la documentation . . . . .         | 3         |
| 1.3      | Contexte technique et justification de l'approche FPGA . . . . . | 4         |
| <b>2</b> | <b>Spécifications fonctionnelles</b>                             | <b>5</b>  |
| 2.1      | Fonctionnalités du système . . . . .                             | 5         |
| 2.2      | Diagramme de flux de données . . . . .                           | 7         |
| <b>3</b> | <b>Design préliminaire</b>                                       | <b>8</b>  |
| 3.1      | Exploration des approches . . . . .                              | 8         |
| 3.2      | Justification de la solution retenue . . . . .                   | 8         |
| 3.3      | Architecture générale . . . . .                                  | 8         |
| <b>4</b> | <b>Design détaillé</b>                                           | <b>11</b> |
| 4.1      | Démarche de développement . . . . .                              | 11        |
| 4.2      | Description des modules . . . . .                                | 11        |
| 4.2.1    | Module 1 — Type de convertisseur . . . . .                       | 11        |
| 4.2.2    | Module 2 — Paramétrage utilisateur . . . . .                     | 12        |
| 4.2.3    | Module 3 — Simulation temps réel sur FPGA . . . . .              | 12        |
| 4.2.4    | Module 4 — Affichage du résultat . . . . .                       | 14        |
| 4.3      | Architecture détaillée et modèle de déploiement . . . . .        | 14        |
| <b>5</b> | <b>Validation</b>                                                | <b>15</b> |
| 5.1      | Plan de tests . . . . .                                          | 15        |
| 5.2      | Validation fonctionnelle sur CPU . . . . .                       | 15        |
| 5.3      | Validation temps réel sur FPGA . . . . .                         | 16        |
| 5.4      | Comparaison CPU vs FPGA . . . . .                                | 17        |
| 5.5      | Analyse selon les spécifications . . . . .                       | 18        |
| <b>6</b> | <b>Conclusion</b>                                                | <b>19</b> |
| 6.1      | Évaluation du système . . . . .                                  | 19        |
| 6.2      | Limites et incertitudes . . . . .                                | 19        |
| 6.3      | Travaux futurs . . . . .                                         | 20        |
| <b>7</b> | <b>Apprentissages et perspectives</b>                            | <b>20</b> |

## Table des figures

|    |                                                                                       |    |
|----|---------------------------------------------------------------------------------------|----|
| 1  | Diagramme de flux de données du système de simulation temps réel sur FPGA. . . .      | 7  |
| 2  | Vue d'ensemble du système FPGA dans Simulink . . . . .                                | 9  |
| 3  | Architecture interne du bloc FPGA. . . . .                                            | 9  |
| 4  | Dashboard MATLAB utilisé pour piloter la simulation en temps réel . . . . .           | 9  |
| 5  | Carte FPGA PYNQ-Z2 utilisée pour l'implémentation. . . . .                            | 10 |
| 6  | Génération automatique du code HDL via HDL Coder. . . . .                             | 10 |
| 7  | Interface de déploiement sur FPGA via la boîte à outils NOA. . . . .                  | 10 |
| 8  | Convertisseurs DC-DC modélisés dans le projet. . . . .                                | 11 |
| 9  | Architecture du générateur PWM conçu pour l'implémentation FPGA. . . . .              | 13 |
| 10 | Architecture du contrôleur PI implanté dans le système. . . . .                       | 13 |
| 11 | Sous-système HDL généré automatiquement par HDL coder. . . . .                        | 14 |
| 12 | Modèle Simulink complet avec le bloc PYNQ-Z2 généré par NOA . . . . .                 | 14 |
| 13 | Validation fonctionnelle des convertisseurs sur CPU. . . . .                          | 15 |
| 14 | Validation fonctionnelle de la simulation temps réel des convertisseurs sur FPGA. . . | 16 |
| 15 | Dashboard MATLAB utilisé pour mesurer les performances de simulation sur CPU .        | 17 |

## Liste des tableaux

|   |                                                                     |    |
|---|---------------------------------------------------------------------|----|
| 1 | Éléments matériels et logiciels du projet. . . . .                  | 10 |
| 2 | Conformité des résultats aux spécifications fonctionnelles. . . . . | 18 |

# 1 Identification du problème

## 1.1 Définition du problème

Les convertisseurs DC-DC (Buck et Boost) sont largement utilisés dans les systèmes électroniques et électriques modernes, notamment dans les alimentations de circuits intégrés, les véhicules électriques et les applications aéronautiques. Leur rôle fondamental est d'adapter une tension d'entrée continue afin de fournir une tension de sortie régulée.

Avant toute implémentation matérielle, il est essentiel de simuler le comportement de ces convertisseurs afin de valider leurs performances dynamiques et leur stabilité avant leur intégration dans un système réel. Cependant, les outils de simulation classiques sur CPU présentent des limitations importantes. Ces simulations sont non déterministes : le temps d'exécution ne correspond pas au temps physique et dépend de la charge instantanée du processeur. À titre d'exemple, simuler  $5 \times 10^{-2}$  s peut nécessiter plusieurs dizaines de minutes sur CPU.

La simulation temps réel sur FPGA répond à cette problématique : le temps de calcul est strictement synchronisé avec le temps physique.

Ce projet démontre cette approche appliquée aux convertisseurs Buck et Boost.

## 1.2 Revue de la littérature et de la documentation

L'utilisation des FPGA (*Field-Programmable Gate Arrays*) est reconnue comme une solution de référence pour la simulation temps réel. Contrairement aux CPU, les FPGA reposent sur une architecture parallèle dans laquelle plusieurs opérations sont exécutées simultanément à chaque cycle d'horloge, ce qui permet de garantir une latence fixe, prévisible et indépendante de la charge du système.

La majorité des travaux existants dans ce domaine reposent sur l'utilisation de langages de description matérielle tels que le VHDL ou le Verilog. Bien que ces approches offrent un contrôle très fin de l'implémentation matérielle, elles nécessitent une expertise spécialisée et augmentent significativement la complexité du développement.

Dans ce projet, une approche alternative est adoptée en utilisant MATLAB/Simulink combiné à la boîte à outils HDL Coder. Cette solution permet de générer automatiquement le code matériel HDL à partir d'un modèle comportemental, sans aucune écriture manuelle de VHDL ou Verilog. Le déploiement sur la carte FPGA PYNQ-Z2 est ensuite réalisé via la boîte à outils NOA, qui permet aux utilisateurs Simulink de déployer des applications FPGA à faible latence sans expertise approfondie en conception matérielle.

De plus, la plateforme PYNQ-Z2 facilite l'intégration avec une interface utilisateur, notamment via l'environnement Jupyter Notebook, en s'appuyant sur des interfaces de communication de type AXI pour l'échange de données entre le FPGA et le système logiciel.

Par ailleurs, les convertisseurs Buck et Boost sont largement étudiés en électronique de puissance. De nombreuses ressources documentaires et exemples d'implémentation sont disponibles, notamment sous MATLAB/Simulink, ce qui facilite leur modélisation et leur validation. Ces documents fournissent également les équations de référence utilisées pour la modélisation sous Simulink-Simscape.

### 1.3 Contexte technique et justification de l'approche FPGA

La condition fondamentale pour qu'une simulation soit qualifiée de temps réel est que le temps de calcul nécessaire à chaque pas de simulation soit inférieur ou égal à la durée de ce pas. Autrement dit, le système doit être capable de terminer tous les calculs avant le prochain instant de simulation.

Cette contrainte s'écrit :

$$T_{\text{calc, FPGA}} \leq T_s$$

où  $T_s$  est le pas de simulation imposé.

Dans ce projet, la carte FPGA PYNQ-Z2 est cadencée à une fréquence de 40 MHz, ce qui correspond à une période d'horloge de :

$$T_{\text{clk}} = \frac{1}{40 \times 10^6} = 25 \text{ ns}$$

Le pas de simulation choisi est :

$$T_s = 50 \text{ ns}$$

Ainsi, chaque pas de simulation correspond à deux cycles d'horloge FPGA. Cela signifie que tous les calculs nécessaires doivent être exécutés en au plus deux cycles d'horloge, ce qui garantit une exécution strictement synchronisée avec le temps physique. Cette contrainte est difficile à respecter sur CPU en raison de l'exécution séquentielle des instructions et de la variabilité du temps de calcul.

À l'inverse, le FPGA permet de satisfaire cette condition grâce à plusieurs propriétés essentielles :

- parallélisme matériel : plusieurs opérations sont exécutées simultanément ;
- latence déterministe : le temps de calcul est fixe et prévisible ;
- horloge dédiée : l'exécution est cadencée indépendamment de la charge du système.

Enfin, l'ensemble de la chaîne de développement est réalisé sous MATLAB/Simulink, avec génération automatique du code HDL via HDL Coder et déploiement sur FPGA via la boîte à outils NOA. Cette approche permet de démontrer la faisabilité d'une simulation temps réel sur FPGA sans recourir à une programmation matérielle bas niveau.

Ce projet constitue ainsi une preuve de concept illustrant l'intérêt des FPGA pour la simulation temps réel de convertisseurs DC-DC.

## 2 Spécifications fonctionnelles

### 2.1 Fonctionnalités du système

Le système développé est une plateforme de simulation en temps réel de convertisseurs DC-DC (Buck et Boost) sur FPGA. Il permet de simuler, contrôler et observer le comportement du convertisseur à l'aide d'une interface utilisateur.

#### Entrées

- Paramètres du convertisseur : fréquence de commutation (10 kHz à 150 kHz), rapport cyclique (0 à 100 %), tension de sortie de référence  $V_{out,ref}$  (0 à 12 V), ainsi que la tension d'entrée  $V_{in}$ . Dans ce projet,  $V_{in}$  est fixée à 12 V pour le convertisseur Buck et à 5 V pour le convertisseur Boost.
- Mode de fonctionnement : boucle ouverte ou boucle fermée avec contrôleur PI.
- Signaux de contrôle : signal **enable** pour l'activation ou la désactivation de la simulation, et signal **reset** pour la réinitialisation du système.
- Modèle Simulink : modèle du convertisseur Buck ou Boost incluant les composants passifs ( $L$ ,  $C$ ,  $R$ ) et la logique de commutation.

#### Sorties

- Tension de sortie simulée  $V_{out}$  en temps réel.
- Scopes et graphiques MATLAB permettant l'analyse des formes d'onde et du comportement dynamique du système.

#### Fonctions

- Simuler le comportement dynamique d'un convertisseur DC-DC Buck ou Boost.
- Analyser la réponse du système, notamment la tension de sortie, le signal PWM et l'erreur de régulation.
- Assurer un contrôle en temps réel de la tension de sortie : lorsque la consigne  $V_{out,ref}$  est modifiée, la sortie  $V_{out}$  doit suivre cette nouvelle consigne.
- Permettre le fonctionnement en boucle ouverte et en boucle fermée avec contrôleur PI.
- Comparer les performances entre une simulation classique sur CPU et une simulation temps réel sur FPGA.

#### Performances requises

- Respect du temps réel avec un pas de simulation  $T_s = 50$  ns et un temps de calcul FPGA vérifiant  $T_{calc} \leq T_s$ .
- Fréquence de commutation configurable jusqu'à 150 kHz.
- Exécution déterministe, avec une latence constante à chaque pas de simulation.
- Modification des paramètres de commande sans recompilation du modèle.
- Stabilité de la réponse du convertisseur lors des changements de consigne.

### Facteurs humains

- Interface simple à utiliser, avec des commandes claires pour modifier la fréquence, le rapport cyclique, la consigne et le mode de fonctionnement.
- Lecture intuitive des résultats grâce aux graphiques MATLAB, aux scopes Simulink et à l'interface Jupyter.
- Compréhension rapide du comportement du système grâce à une visualisation directe de  $V_{out}$ , du PWM et de l'erreur du contrôleur.
- Interaction facilitée avec le système, permettant à l'utilisateur de modifier les paramètres et d'observer immédiatement l'effet sur la simulation.

### Gestion des erreurs

- Arrêt immédiat de la simulation lorsque le signal **enable** est désactivé.
- Réinitialisation du système lorsque le signal **reset** est activé.

## 2.2 Diagramme de flux de données

Le diagramme de flux de données ci-dessous décrit la circulation des informations dans le système. Les paramètres utilisateur ( $V_{ref}$ , fréquence, rapport cyclique, mode) alimentent le bloc de contrôle et le générateur PWM. Ces deux blocs pilotent le modèle FPGA (HDL Subsystem), qui calcule  $V_{out}$ . Ces grandeurs alimentent à leur tour le contrôleur PI (boucle fermée) et le bloc de visualisation, dont les résultats sont affichés en temps réel.

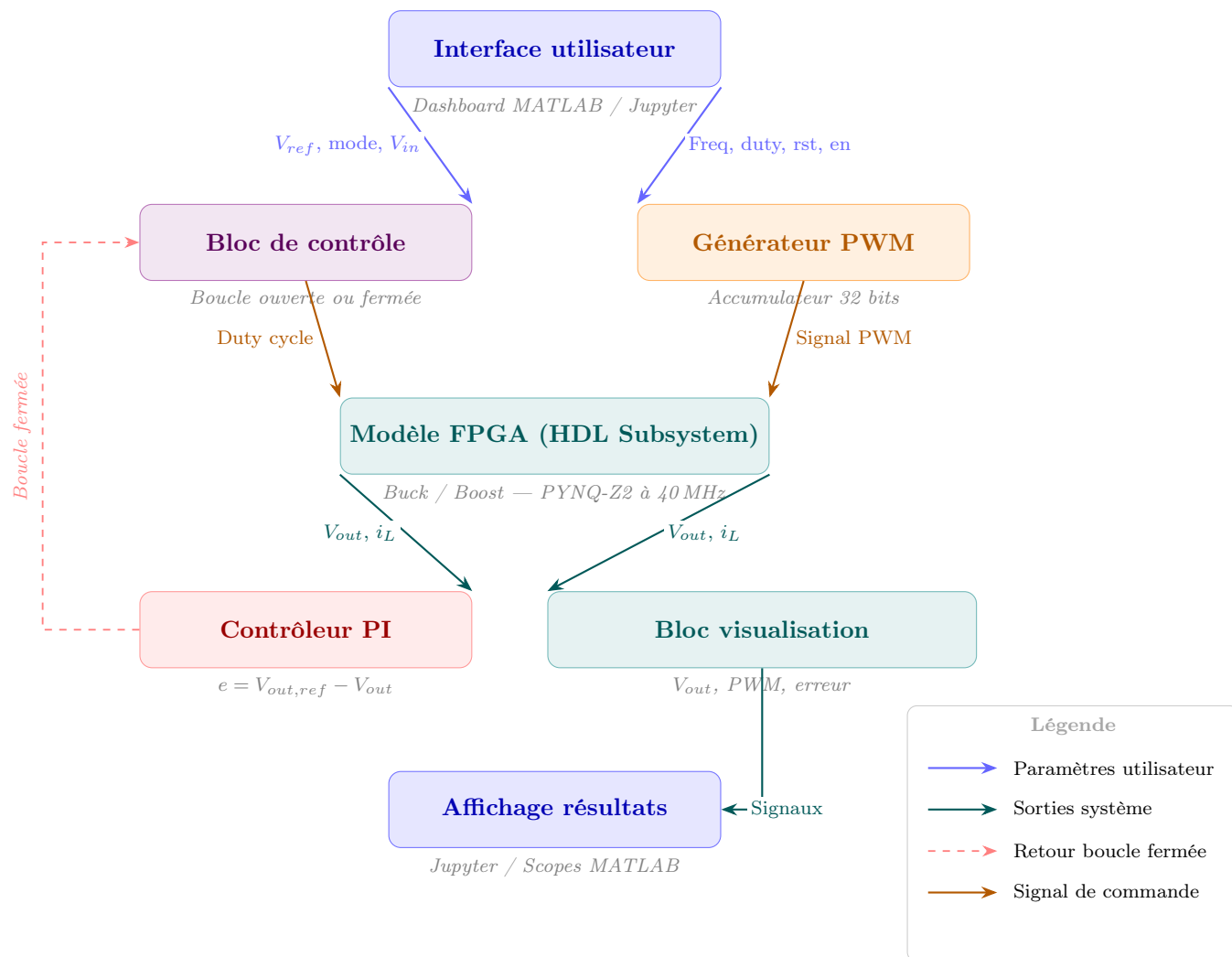


FIGURE 1 – Diagramme de flux de données du système de simulation temps réel sur FPGA.

## 3 Design préliminaire

### 3.1 Exploration des approches

Deux approches d'implémentation sur FPGA ont été envisagées.

- **Développement HDL manuel via Vivado** : modélisation directe en VHDL/Verilog et génération du bitstream via Vivado. Cette méthode s'est révélée difficilement accessible dans le cadre de ce projet : la transition entre la modélisation MATLAB et la programmation HDL bas niveau représente un obstacle technique significatif qui aurait considérablement alourdi le cycle de développement.
- **Génération automatique via MATLAB/Simulink et HDL Coder** : le modèle Simulink est converti automatiquement en HDL synthétisable, puis déployé sur FPGA depuis MATLAB via la boîte à outils NOA, sans aucune écriture manuelle de VHDL ou Verilog.

La seconde approche a été retenue.

### 3.2 Justification de la solution retenue

L'approche MATLAB/Simulink avec HDL Coder constitue un compromis optimal entre accessibilité et performances. Elle présente les avantages suivants par rapport au développement HDL manuel.

- Simplicité de développement : pas d'écriture manuelle en VHDL/Verilog, la conception reste au niveau des blocs fonctionnels Simulink.
- Rapidité du cycle de développement : la génération automatique du code HDL et le déploiement sur le FPGA via la boîte à outils NOA sont entièrement réalisés depuis MATLAB, ce qui permet de réduire considérablement le temps de développement tout en facilitant les itérations de conception.
- Conservation des avantages FPGA : malgré l'abstraction, le système bénéficie pleinement du parallélisme matériel, de la latence déterministe et de l'exécution en temps réel.

### 3.3 Architecture générale

La figure 2 présente la vue d'ensemble du modèle Simulink implémenté sur FPGA. Le bloc FPGA encapsule l'ensemble de la logique matérielle du système. Il reçoit en entrée les paramètres utilisateur, notamment les signaux de contrôle (**reset**, **enable**), le rapport cyclique, la fréquence, le mode de contrôle (boucle ouverte ou fermée) ainsi que la consigne  $V_{out,ref}$ .

En sortie, le système fournit la tension simulée  $V_{out}$ , le signal PWM ainsi que l'erreur du contrôleur. Ces grandeurs sont visualisées à l'aide de scopes, permettant une analyse du comportement dynamique du convertisseur en temps réel.

Cette architecture est identique pour les convertisseurs Buck et Boost : seul le sous-système HDL interne, correspondant au modèle du convertisseur, diffère.

La figure 3 illustre l'architecture interne du bloc FPGA. On y retrouve les principaux sous-modules : le sélecteur de mode (boucle ouverte / fermée), le générateur PWM, le sous-système HDL représentant le modèle du convertisseur, les blocs de conversion de données ainsi que le contrôleur PI.

La figure 4 présente le dashboard MATLAB utilisé pour piloter la simulation en temps réel. Cette interface permet à l'utilisateur de modifier dynamiquement les paramètres du système (rapport cyclique, fréquence, consigne  $V_{ref}$ , mode de fonctionnement) sans nécessiter de recompilation du modèle.

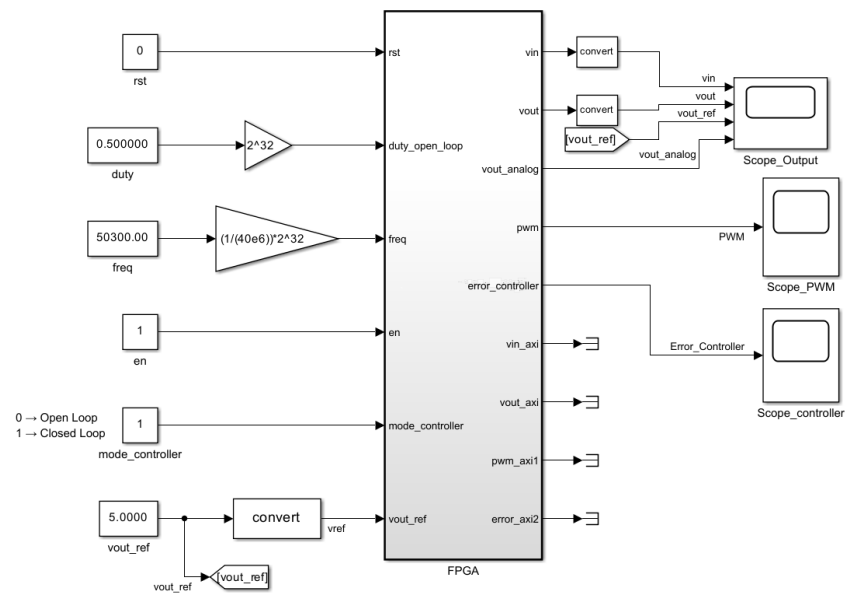


FIGURE 2 – Vue d'ensemble du système FPGA dans Simulink

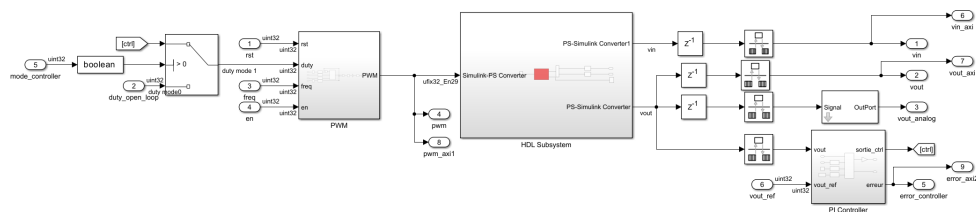


FIGURE 3 – Architecture interne du bloc FPGA.

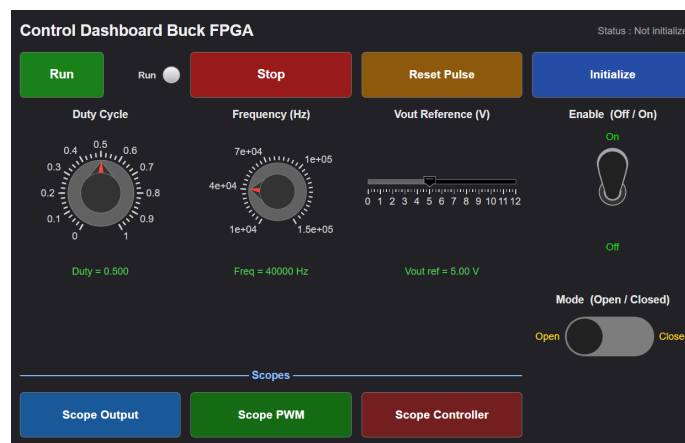


FIGURE 4 – Dashboard MATLAB utilisé pour piloter la simulation en temps réel

TABLE 1 – Éléments matériels et logiciels du projet.

| Élément                    | Rôle                       | Outil / Plateforme |
|----------------------------|----------------------------|--------------------|
| PYNQ-Z2 (Xilinx Zynq-7000) | Exécution temps réel       | Carte FPGA         |
| MATLAB/Simulink-Simscape   | Modélisation               | MathWorks          |
| HDL Coder                  | Génération HDL automatique | MathWorks          |
| NOA Toolbox                | Déploiement sur FPGA       | MathWorks          |

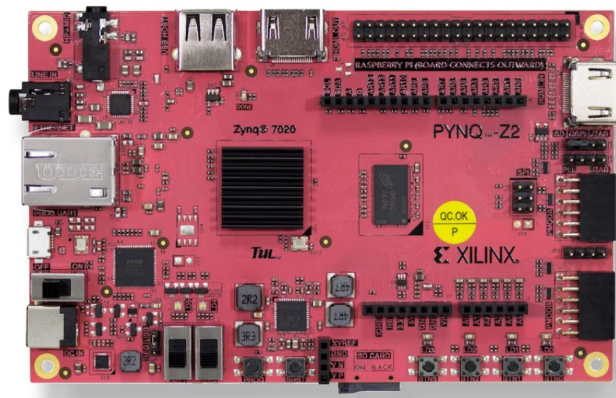


FIGURE 5 – Carte FPGA PYNQ-Z2 utilisée pour l'implémentation.

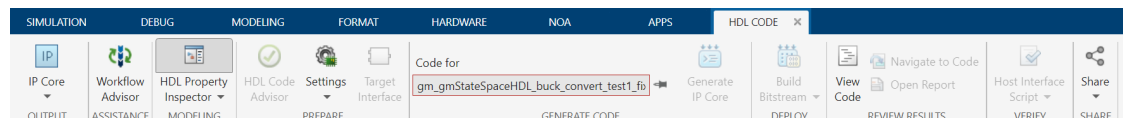


FIGURE 6 – Génération automatique du code HDL via HDL Coder.

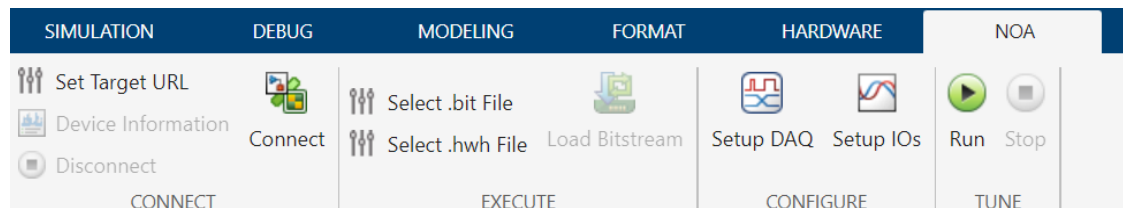


FIGURE 7 – Interface de déploiement sur FPGA via la boîte à outils NOA.

## 4 Design détaillé

### 4.1 Démarche de développement

Le système a d'abord été validé sur CPU, puis migré vers le FPGA selon les étapes suivantes :

1. Modélisation du convertisseur Buck et Boost sous MATLAB/Simulink-Simscape et validation sur CPU.
2. Conversion automatique en HDL Subsystem via HDL Coder.
3. Ajout des blocs complémentaires : générateur PWM, contrôleur PI, blocs d'affichage.
4. Génération finale du modèle HDL complet.
5. Déploiement sur FPGA via la boîte à outils NOA.

L'ensemble est généré sans aucune écriture manuelle de VHDL ou Verilog.

### 4.2 Description des modules

Le système est organisé en quatre modules fonctionnels.

#### 4.2.1 Module 1 — Type de convertisseur

Ce module constitue le cœur physique du système. Il contient le modèle mathématique du convertisseur Buck ou Boost implémenté sous Simulink-Simscape. Les paramètres de conception retenus sont :

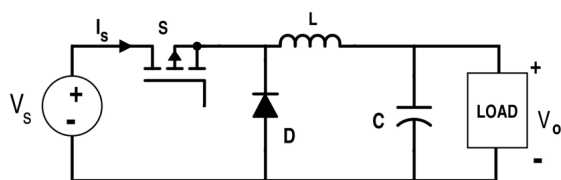
- Convertisseur Buck :  $V_{in} = 12 \text{ V}$
- Convertisseur Boost :  $V_{in} = 5 \text{ V}$
- Charge :  $R = 10 \Omega$
- Inductance :  $L = 100 \mu\text{H}$
- Capacité :  $C = 47 \mu\text{F}$
- Rapport cyclique nominal :  $D = 40\%$
- Fréquence de découpage nominale :  $f_{sw} = 100 \text{ kHz}$

Les relations théoriques de référence sont :

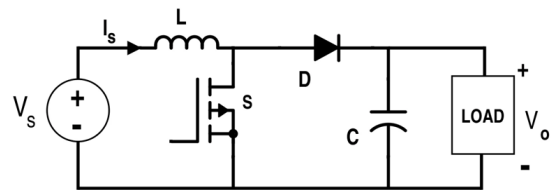
$$V_{out} = D V_{in} \quad (\text{Buck})$$

$$V_{out} = \frac{V_{in}}{1 - D} \quad (\text{Boost})$$

Les valeurs de  $L$ ,  $C$  et  $R$  ont été choisies afin d'obtenir un compromis entre rapidité de réponse, stabilité et faible ondulation.



(a) Topologie Buck.



(b) Topologie Boost.

FIGURE 8 – Convertisseurs DC-DC modélisés dans le projet.

#### 4.2.2 Module 2 — Paramétrage utilisateur

Ce module permet à l'utilisateur de configurer dynamiquement les paramètres de fonctionnement du convertisseur.

Les paramètres configurables sont :

- la tension de référence  $V_{out,ref}$ , utilisée comme consigne pour la régulation en boucle fermée ;
- la fréquence de commutation  $f_{sw}$ , ajustable typiquement entre 10 kHz et 150 kHz ;
- le rapport cyclique  $D$ , utilisé directement comme commande en mode boucle ouverte ;
- le mode de contrôle :
  - boucle ouverte : le convertisseur est piloté uniquement par le rapport cyclique imposé ;
  - boucle fermée : un contrôleur PI ajuste automatiquement la commande afin que  $V_{out}$  suive la consigne  $V_{out,ref}$ .
- le signal **enable**, permettant d'activer ou de désactiver l'exécution du système ;
- le signal **reset**, utilisé pour réinitialiser les états internes du système.

Ces paramètres sont introduits via le dashboard MATLAB et peuvent être modifiés en temps réel sans recompilation du modèle.

Avant leur transmission au FPGA, certains paramètres doivent être convertis vers des représentations numériques compatibles avec l'implémentation matérielle :

- le rapport cyclique est mis à l'échelle sur 32 bits pour alimenter le générateur PWM ;
- la fréquence de commutation est convertie en pas d'incrément pour l'accumulateur numérique du PWM ;
- la consigne  $V_{ref}$  est convertie dans le format numérique utilisé par le contrôleur PI.

#### 4.2.3 Module 3 — Simulation temps réel sur FPGA

Ce module constitue le cœur de l'implémentation. Il exécute le modèle du convertisseur en temps réel sur le FPGA PYNQ-Z2, cadencé à 40 MHz, soit une période d'horloge de 25 ns. Avec un pas de simulation fixé à 50 ns, chaque itération est exécutée en deux cycles d'horloge, ce qui permet une exécution déterministe synchronisée avec le temps physique.

Le module FPGA regroupe plusieurs sous-modules interconnectés :

- **Sous-module de sélection de mode (switch)** Il permet de sélectionner le mode de fonctionnement :
  - boucle ouverte : le rapport cyclique est imposé directement par l'utilisateur ;
  - boucle fermée : la commande provient du contrôleur PI.
- **Sous-module générateur PWM** Génère le signal de commande numérique destiné à piloter l'interrupteur du convertisseur.
- **Sous-module HDL Subsystem** Contient le modèle matériel du convertisseur Buck ou Boost généré automatiquement par HDL Coder.
- **Sous-module contrôleur PI** Réalise la régulation de la tension de sortie en mode boucle fermée.

L'ensemble de ces sous-modules est exécuté de manière synchrone et parallèle sur FPGA.

**Générateur PWM** Les blocs PWM standards de Simulink n'étant pas compatibles avec la synthèse HDL, un générateur PWM dédié de résolution 32 bits a été conçu.

Son principe repose sur un accumulateur numérique :

$$\text{freq\_step} = \frac{f_{sw}}{f_{clk}} 2^{32}$$

À chaque cycle d'horloge, l'accumulateur s'incrémente de cette valeur. Le signal PWM est obtenu en comparant sa valeur au seuil correspondant au rapport cyclique :

$$PWM = \begin{cases} 1 & \text{si compteur} < Duty \\ 0 & \text{sinon} \end{cases}$$

Cette architecture permet de générer un PWM compatible HDL, à fréquence ajustable et haute résolution.

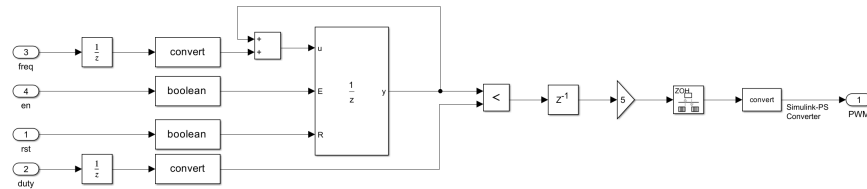


FIGURE 9 – Architecture du générateur PWM conçu pour l'implémentation FPGA.

**Contrôleur PI** En mode boucle fermée, le rapport cyclique est ajusté automatiquement par un contrôleur PI selon :

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau$$

avec

$$e(t) = V_{out,ref} - V_{out}$$

où :

- $K_p$  améliore la rapidité de réponse ;
- $K_i$  réduit l'erreur statique.

Une saturation en sortie impose des limites physiques au rapport cyclique afin de garantir un fonctionnement réaliste du convertisseur.

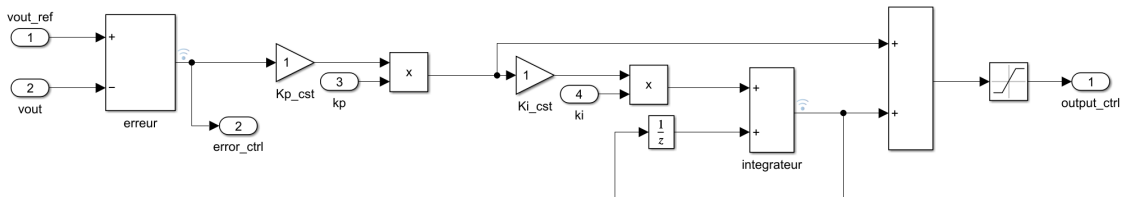


FIGURE 10 – Architecture du contrôleur PI implanté dans le système.

**Sous-système HDL du convertisseur** Le sous-système HDL représente le modèle matériel du convertisseur généré automatiquement à partir du modèle Simulink-Simscape via HDL Coder. Il contient la dynamique discrète du convertisseur ainsi que les blocs mémoire et délais nécessaires à l'évolution des variables d'état.

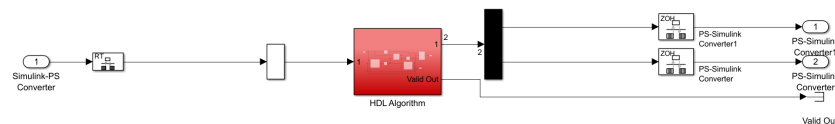


FIGURE 11 – Sous-système HDL généré automatiquement par HDL coder.

#### 4.2.4 Module 4 — Affichage du résultat

Les grandeurs  $V_{in}$ ,  $V_{out}$ , le signal PWM et l'erreur  $e$  sont visualisées via des scopes Simulink et une interface Jupyter.

### 4.3 Architecture détaillée et modèle de déploiement

La figure 12 montre le modèle Simulink complet avec le bloc PYNQ-Z2 généré par NOA (en turquoise), qui encapsule l'ensemble des modules précédents.

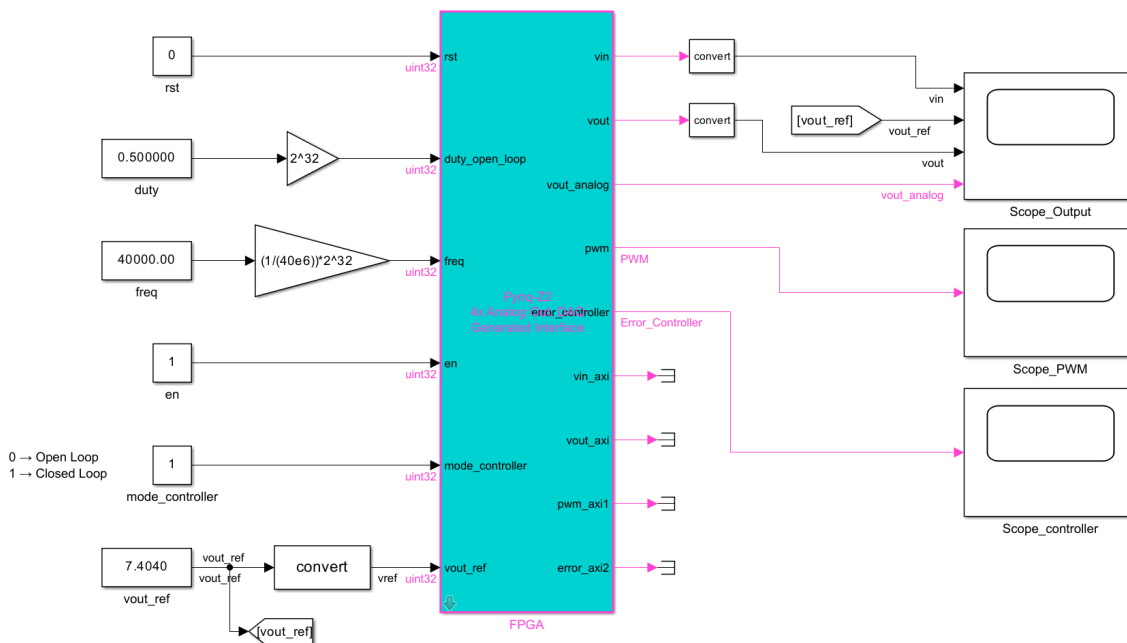


FIGURE 12 – Modèle Simulink complet avec le bloc PYNQ-Z2 généré par NOA

## 5 Validation

### 5.1 Plan de tests

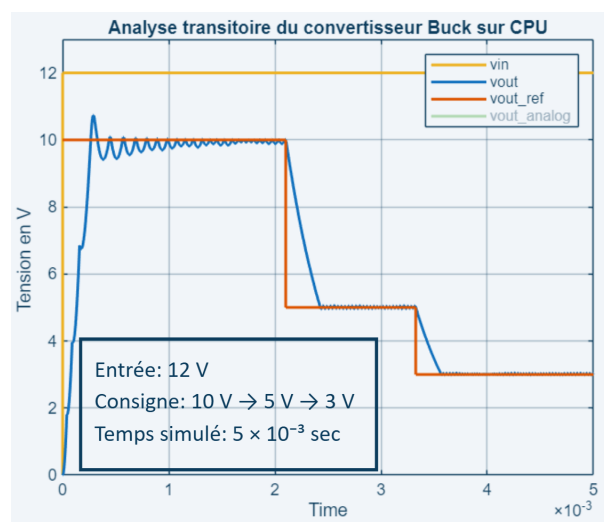
La validation repose sur trois niveaux complémentaires :

- CPU : vérification que  $V_{out}$  suit correctement les consignes et que les formes d'onde correspondent aux équations théoriques.
- Modules : validation indépendante de chaque bloc (PWM, PI, convertisseur) via des scopes dédiés.
- Système FPGA : vérification de l'exécution en temps réel, de la cohérence des signaux et du gain de performance par rapport au CPU.

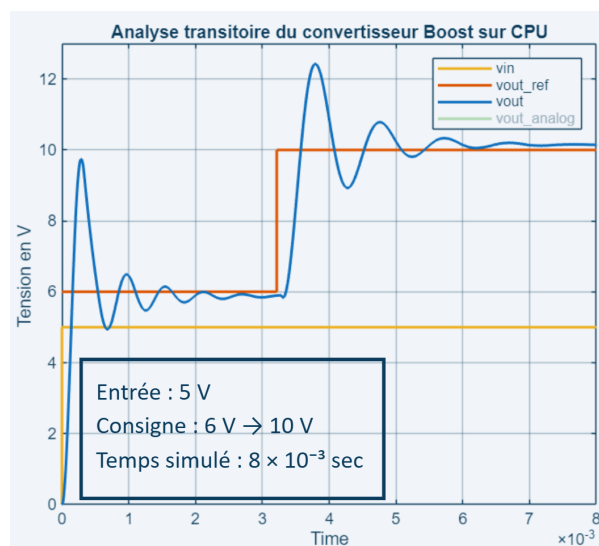
### 5.2 Validation fonctionnelle sur CPU

**Buck.** Testé avec  $V_{in} = 12\text{ V}$  et des consignes successives de  $10\text{ V} \rightarrow 5\text{ V} \rightarrow 3\text{ V}$  (figure 13a). La tension de sortie suit chaque consigne avec un régime transitoire amorti conforme à la théorie.

**Boost.** Testé avec  $V_{in} = 5\text{ V}$  et des consignes de  $6\text{ V} \rightarrow 10\text{ V}$  (figure 13b). La tension converge vers chaque consigne.



(a) Analyse transitoire du Buck sur CPU



(b) Analyse transitoire du Boost sur CPU

FIGURE 13 – Validation fonctionnelle des convertisseurs sur CPU.

### 5.3 Validation temps réel sur FPGA

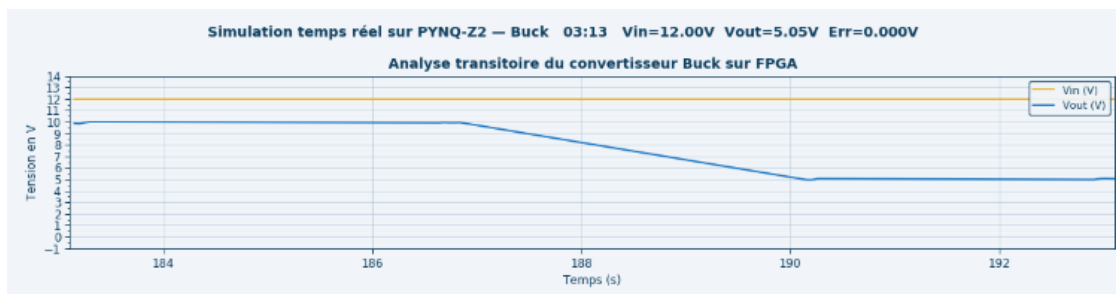
Les figures 14a et 14b présentent les résultats affichés en temps réel via Jupyter depuis la carte PYNQ-Z2. L'axe temporel en secondes correspond ici directement au temps physique réel écoulé, ce qui constitue une validation du caractère temps réel de l'exécution.

**Buck.** Le convertisseur Buck a été testé avec  $V_{in} = 12\text{ V}$  et des changements successifs de consigne de 10 V vers 5 V. La tension de sortie suit correctement chaque nouvelle consigne, validant le fonctionnement du contrôleur en boucle fermée.

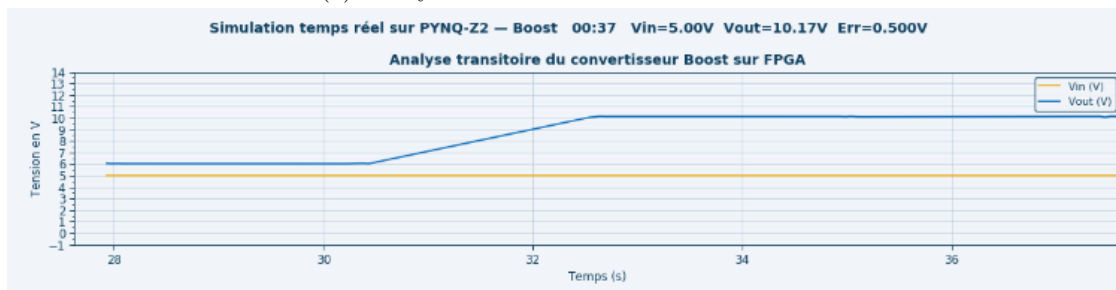
On observe que la simulation s'est exécutée pendant 3 min 13 s sur Jupyter, et cette durée a effectivement correspondu à 3 min 13 s en temps réel. Autrement dit, le système n'a pas simulé cette durée plus rapidement ou plus lentement : l'exécution est restée synchronisée avec le temps physique, ce qui constitue une démonstration directe du temps réel.

**Boost.** Le convertisseur Boost a été testé avec  $V_{in} = 5\text{ V}$  et des consignes successives de 6 V vers 10 V. La tension converge vers chaque consigne avec une réponse stable.

De même, la simulation a fonctionné pendant 37 s sur l'interface Jupyter, et cette durée a également correspondu à 37 s mesurées en temps réel. Cette correspondance confirme à nouveau que l'implémentation respecte la contrainte d'exécution temps réel.



(a) Analyse transitoire du Buck sur FPGA



(b) Analyse transitoire du Boost sur FPGA

FIGURE 14 – Validation fonctionnelle de la simulation temps réel des convertisseurs sur FPGA.

## 5.4 Comparaison CPU vs FPGA

La figure 15 présente les résultats obtenus avec le dashboard de comparaison CPU. Pour simuler seulement

$5 \times 10^{-4} \text{ s}$

de temps physique, le CPU nécessite 46,4 s de calcul réel, soit un facteur de ralentissement mesuré de

92 778×

Cela signifie que le CPU est incapable de suivre le temps physique pour ce pas de simulation très faible.

Cette limitation provient de la nature séquentielle du CPU : les opérations sont exécutées successivement et le solveur doit résoudre numériquement les équations du convertisseur à chaque pas, ce qui devient extrêmement coûteux lorsque la dynamique est rapide.

À l'inverse, le FPGA exécute le modèle en logique matérielle parallèle. Plusieurs opérations sont réalisées simultanément à chaque cycle d'horloge, avec une latence fixe. Il n'y a donc pas de ralentissement : l'exécution reste synchronisée avec le temps physique.

Au-delà du temps de calcul, cette approche CPU entraîne également une consommation énergétique importante. L'estimation projetée donne :

1159 Wh

pour simuler 1 s de système, ce qui met en évidence l'inefficacité énergétique potentielle d'une simulation fine sur CPU pour ce type d'application.

Cette comparaison montre donc un double avantage du FPGA :

- accélération massive de l'exécution ;
- réduction potentielle de l'énergie nécessaire à la simulation.

Ces résultats confirment l'intérêt du FPGA pour les systèmes à dynamique rapide où les méthodes CPU classiques deviennent rapidement prohibitives.

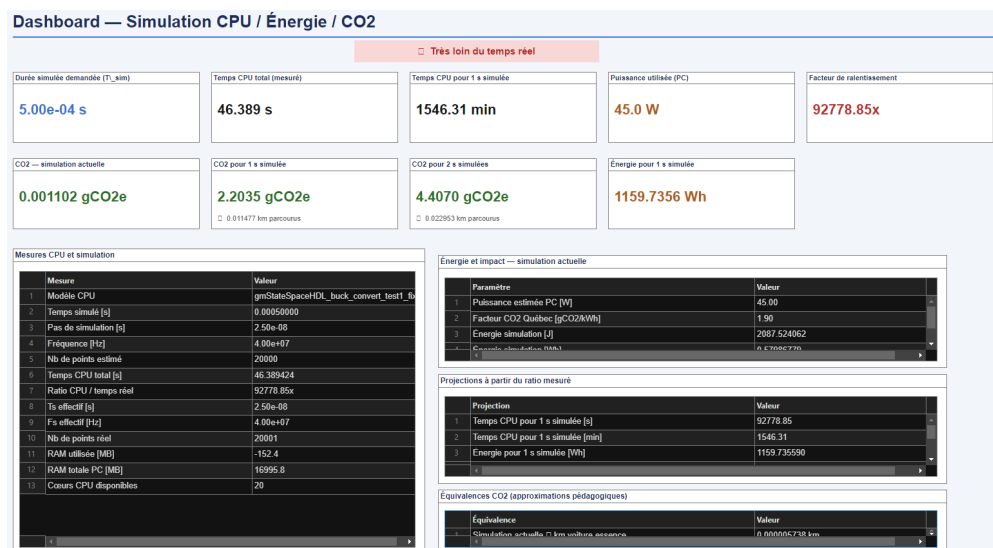


FIGURE 15 – Dashboard MATLAB utilisé pour mesurer les performances de simulation sur CPU

## 5.5 Analyse selon les spécifications

TABLE 2 – Conformité des résultats aux spécifications fonctionnelles.

| Critère                | Spécification   | Résultat             | Statut |
|------------------------|-----------------|----------------------|--------|
| Simulation Buck/Boost  | Fonctionnelle   | Conforme             | ✓      |
| Suivi de consigne      | Correct         | Conforme CPU et FPGA | ✓      |
| Pas de simulation      | 50 ns           | 50 ns respecté       | ✓      |
| Exécution temps réel   | Requise         | Oui sur FPGA         | ✓      |
| Fréquence de découpage | Jusqu'à 150 kHz | Supportée            | ✓      |
| Génération PWM         | Stable          | Conforme             | ✓      |
| Contrôle boucle fermée | Fonctionnel     | Conforme (PI)        | ✓      |
| Gain CPU/FPGA          | Significatif    | $> 66\,000\times$    | ✓      |

## 6 Conclusion

### 6.1 Évaluation du système

Ce projet a permis de démontrer la faisabilité de la simulation en temps réel de convertisseurs DC-DC sur FPGA à partir d'un flot de développement entièrement basé sur MATLAB/Simulink, HDL Coder et la boîte à outils NOA, sans programmation manuelle en VHDL ou Verilog. Cette approche constitue un résultat important du projet, puisqu'elle montre qu'il est possible d'exploiter les avantages du matériel reconfigurable tout en conservant un niveau d'abstraction élevé.

Les objectifs initiaux du projet ont été atteints. Les modèles Buck et Boost ont été modélisés, validés sur CPU puis déployés sur la carte PYNQ-Z2 avec une exécution déterministe en temps réel.

Au-delà des performances, le projet met en évidence l'intérêt des FPGA pour la simulation en temps réel de systèmes dynamiques rapides, particulièrement lorsque les solveurs CPU deviennent prohibitifs en temps de calcul. La comparaison CPU vs FPGA a montré non seulement un avantage majeur en rapidité, mais également un potentiel gain en efficacité énergétique grâce à l'architecture parallèle du FPGA.

Ce projet constitue ainsi une preuve de concept convaincante de simulation temps réel sur FPGA appliquée à l'électronique de puissance.

### 6.2 Limites et incertitudes

Malgré les résultats obtenus, plusieurs limites du prototype doivent être soulignées.

- Affichage et instrumentation : L'observation des signaux via Jupyter constitue une solution pratique de démonstration, mais reste limitée comparativement à une visualisation directe sur oscilloscope ;
- Réglage du contrôleur PI : Les performances en boucle fermée demeurent fortement dépendantes du choix des gains  $K_p$  et  $K_i$ . Un réglage sous-optimal peut dégrader la réponse dynamique, provoquer du dépassement ou affecter la stabilité.
- Cycle de développement HDL : Toute modification du modèle Simulink implique une nouvelle génération HDL, une resynthèse et un redéploiement sur FPGA, ce qui allonge les itérations de conception.
- Performance à basse fréquence de commutation : Le système s'est montré moins performant à basse fréquence de commutation. Ce comportement est lié aux valeurs de  $R$ ,  $L$  et  $C$  choisies initialement, optimisées principalement autour du point nominal de fonctionnement. Une autre sélection de composants ou une adaptation des paramètres pourrait améliorer les performances sur une plage de fréquences plus large.

Ces limites n'invalident pas les résultats, mais définissent plutôt des axes naturels d'amélioration pour une version plus avancée du système.

### 6.3 Travaux futurs

Plusieurs prolongements naturels de ce projet peuvent être envisagés :

- Intégrer une carte FPGA disposant d'ADC/DAC embarqués, par exemple Eclipse Z7, afin d'observer directement les signaux analogiques en temps réel sur oscilloscope.
- Étendre le projet vers une plateforme *Hardware-in-the-Loop* (HIL), par exemple avec l'intégration d'un microcontrôleur comme un STM32 interagissant avec le modèle temps réel implanté sur FPGA.
- Explorer des stratégies de contrôle plus avancées, notamment des approches d'intelligence artificielle telles que l'apprentissage par renforcement pour l'adaptation automatique du contrôle.
- Étendre l'approche à d'autres topologies de conversion : Buck-Boost, Ćuk ou convertisseurs plus complexes.
- Développer une interface utilisateur plus robuste pour le pilotage, la visualisation et l'acquisition de données en temps réel.
- Étudier le comportement des convertisseurs sur une plage plus large de fréquences de commutation et optimiser les paramètres du modèle pour améliorer la robustesse.

Ces travaux futurs pourraient transformer cette preuve de concept en une véritable plateforme de simulation temps réel et de prototypage avancé pour l'électronique de puissance.

## 7 Apprentissages et perspectives

Ce projet a permis d'acquérir des connaissances importantes en modélisation sous Simulink-Simscape, génération automatique de HDL, architecture FPGA, contraintes temps réel et méthodologies de déploiement matériel.

Il a également permis de mieux comprendre la différence fondamentale entre calcul séquentiel sur CPU et exécution parallèle déterministe sur FPGA, ainsi que les compromis pratiques entre abstraction logicielle et implémentation matérielle.

Au-delà des aspects techniques, ce projet a constitué une première expérience concrète de conception complète d'un système FPGA allant de la modélisation jusqu'à la validation expérimentale, tout en ouvrant des perspectives futures en simulation temps réel, électronique de puissance et systèmes reconfigurables.

## Références

- [1] MathWorks, *Hardware-in-the-Loop Implementation of Simscape Model on Speedgoat FPGA I/O Modules*, 2022.
- [2] MathWorks, *Model Simscape Synchronous Buck Converter for Real-Time Simulation and Deployment*, 2022.
- [3] MathWorks, *HDL Coder*, 2026.
- [4] MathWorks, *Simscape Electrical Documentation*, 2026.
- [5] Xilinx/Digilent, *PYNQ-Z2 Board Reference Manual*, 2021.
- [6] MathWorks, *NOA Toolbox — Network-Oriented Approach for FPGA Deployment*, 2026.
- [7] Département de génie électrique, Polytechnique Montréal, *Notes de cours ELE3000*, 2026.